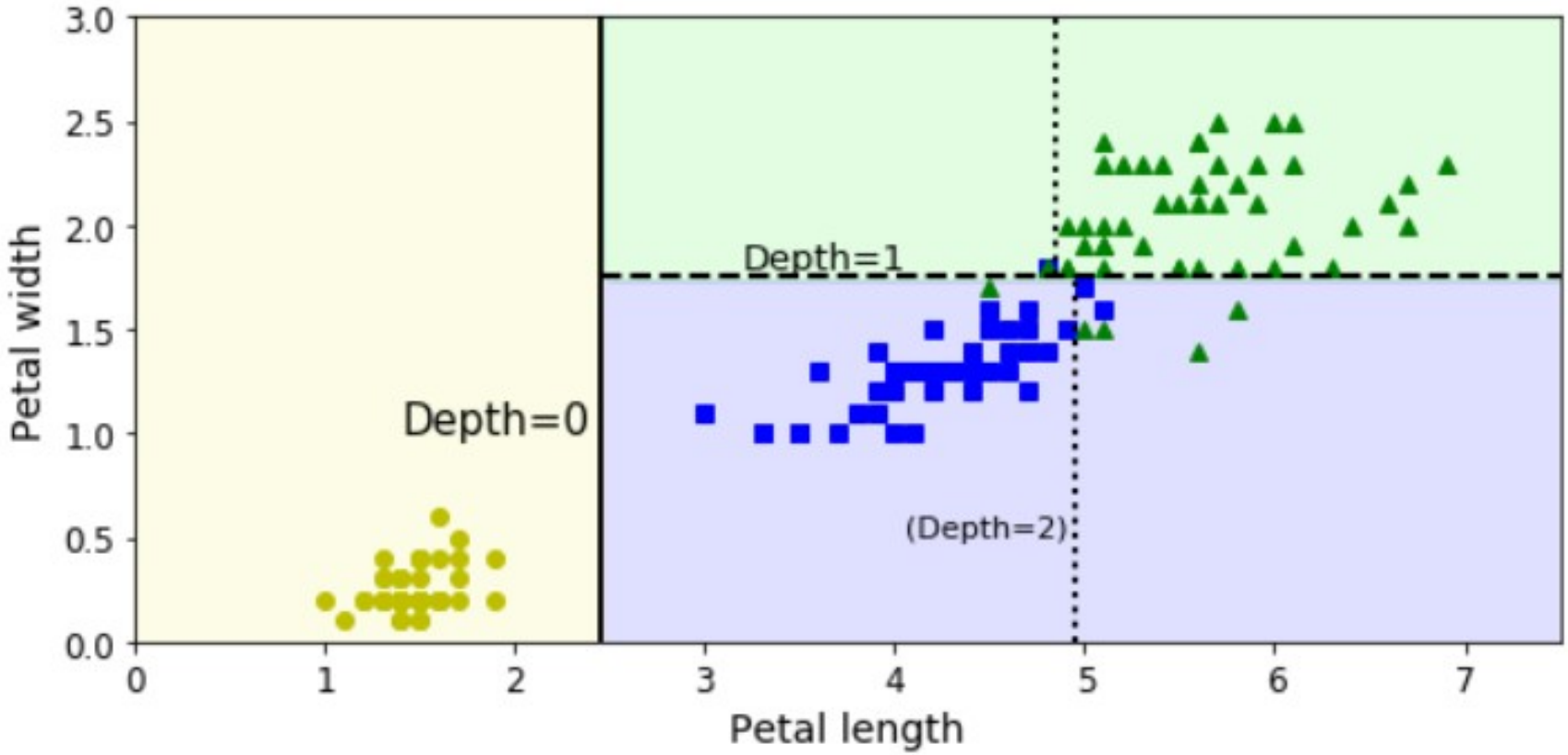
7 de novembro, 2020

Árvores de Decisão

Aldebaro Klautau
Jniversidade Federal do Pará
LASSE

Exemplo: Dataset Iris



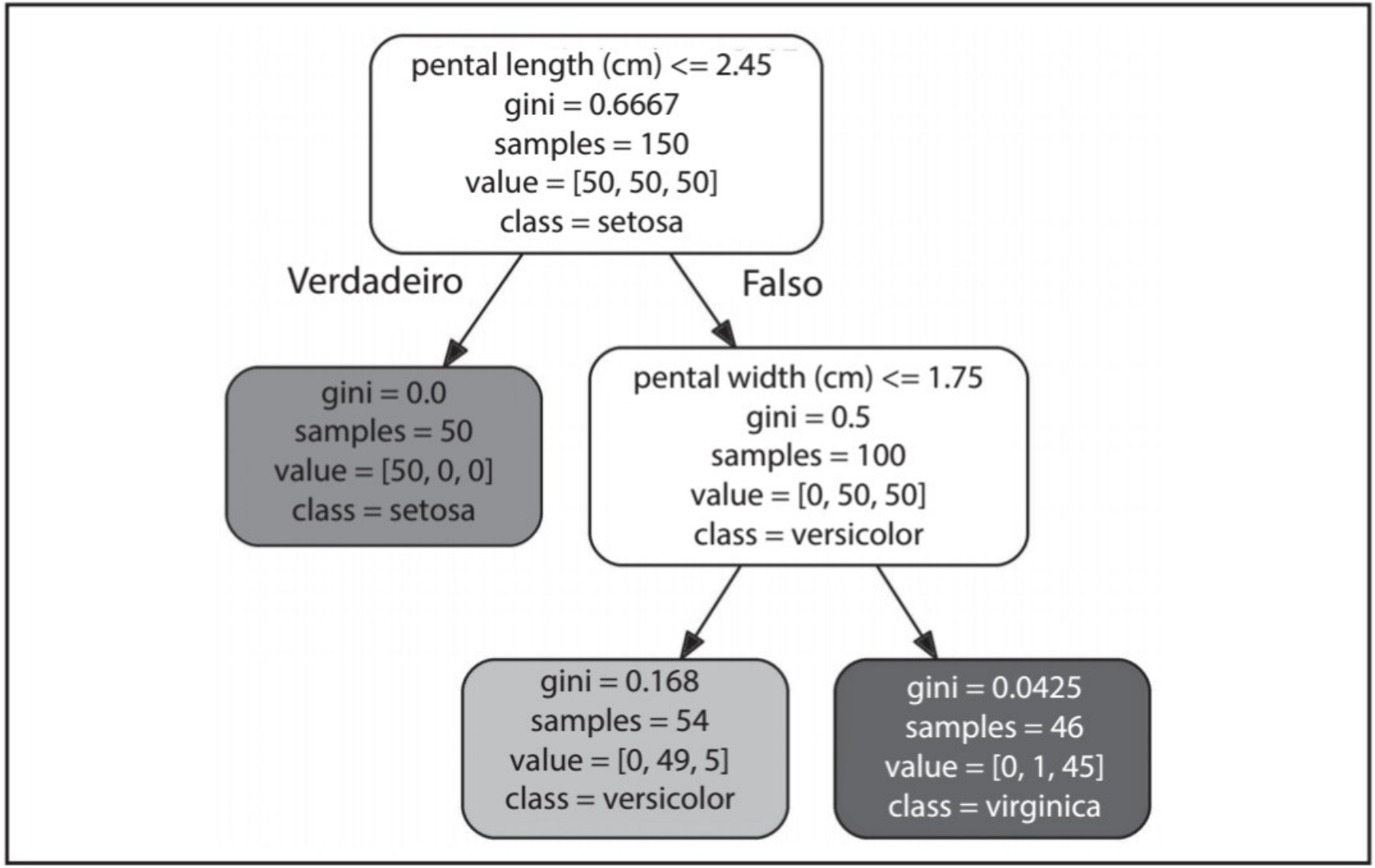
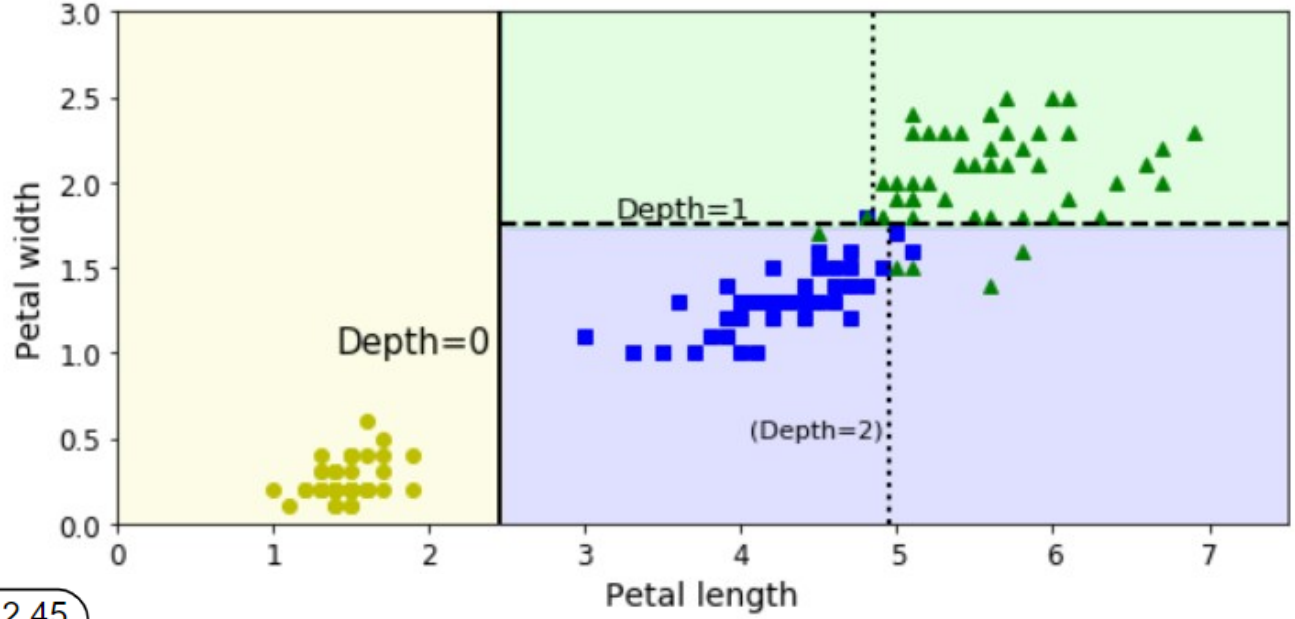
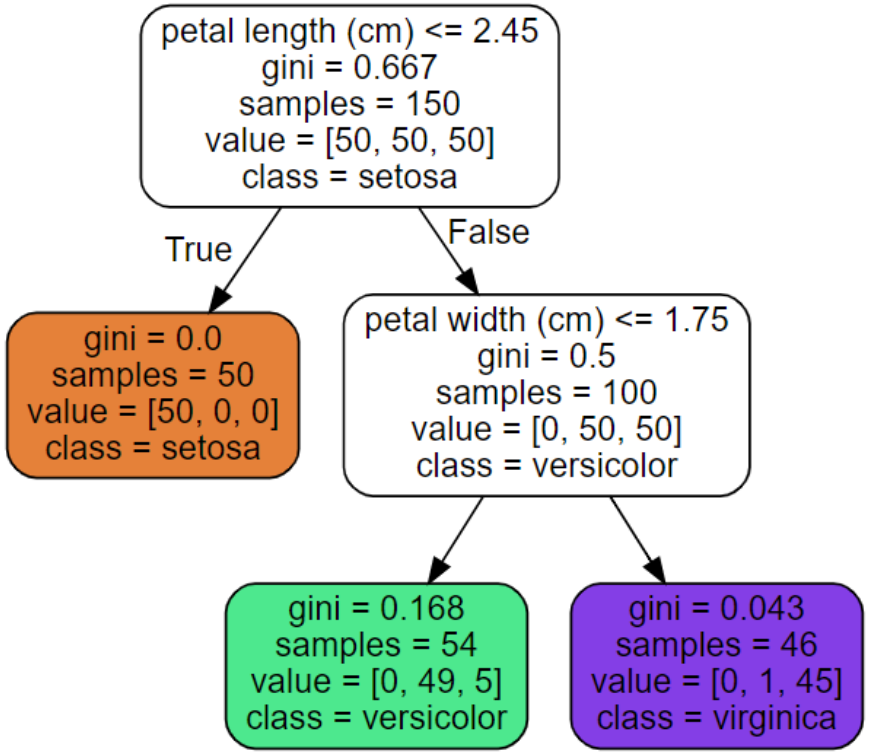


Figura 6-1. Árvore de Decisão da íris


Depth=0
Depth=1
(Depth=2)
Petal width
Petal length
0.0
0.5
1.0
1.5
2.0
2.5
3.0
0
1
2
3
4
5
6
7


petal length (cm) <= 2.45
gini = 0.667
samples = 150
value = [50, 50, 50]
class = setosa
True
False
gini = 0.0
samples = 50
value = [50, 0, 0]
class = setosa
petal width (cm) <= 1.75
gini = 0.5
samples = 100
value = [0, 50, 50]
class = versicolor
gini = 0.168
samples = 54
value = [0, 49, 5]
class = versicolor
gini = 0.043
samples = 46
value = [0, 1, 45]
class = virginica

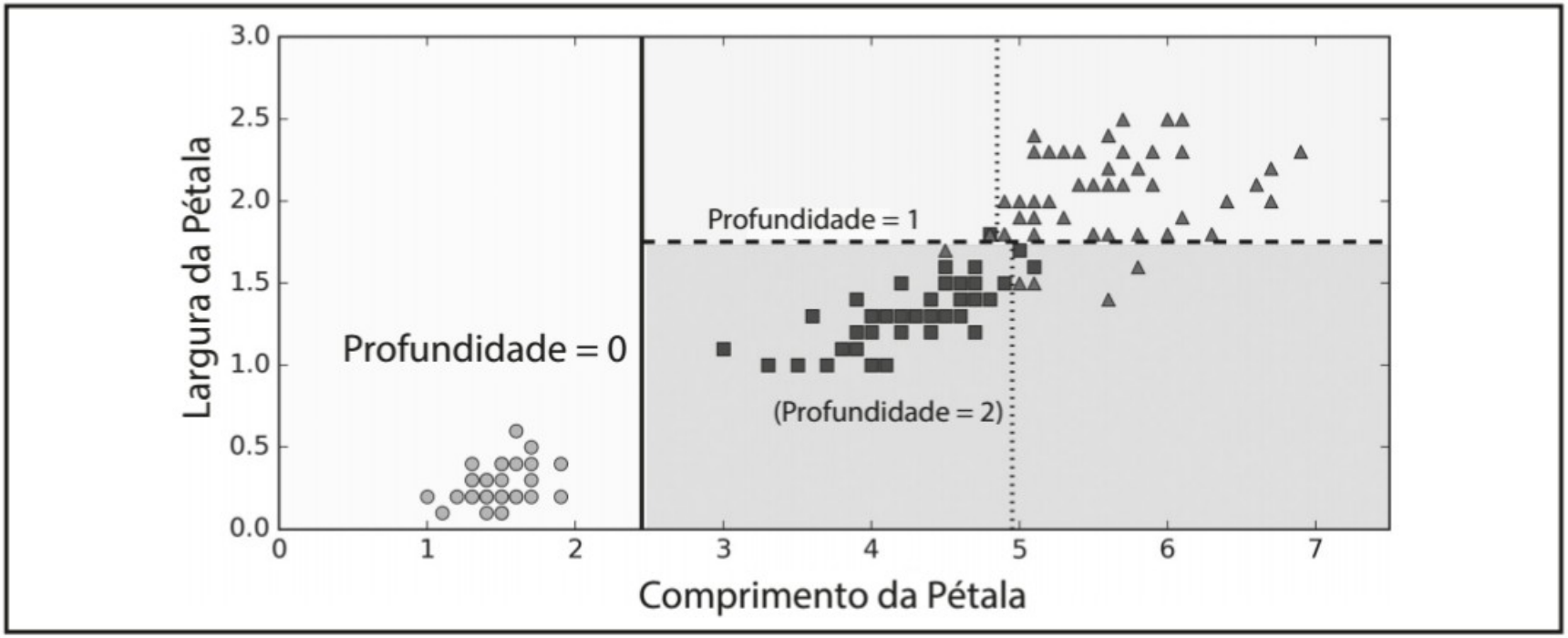


Figura 6-2. Fronteiras de decisão da Árvore de Decisão

Árvores: ótima interpretabilidade

Interpretação do Modelo: Caixa Branca versus Caixa Preta

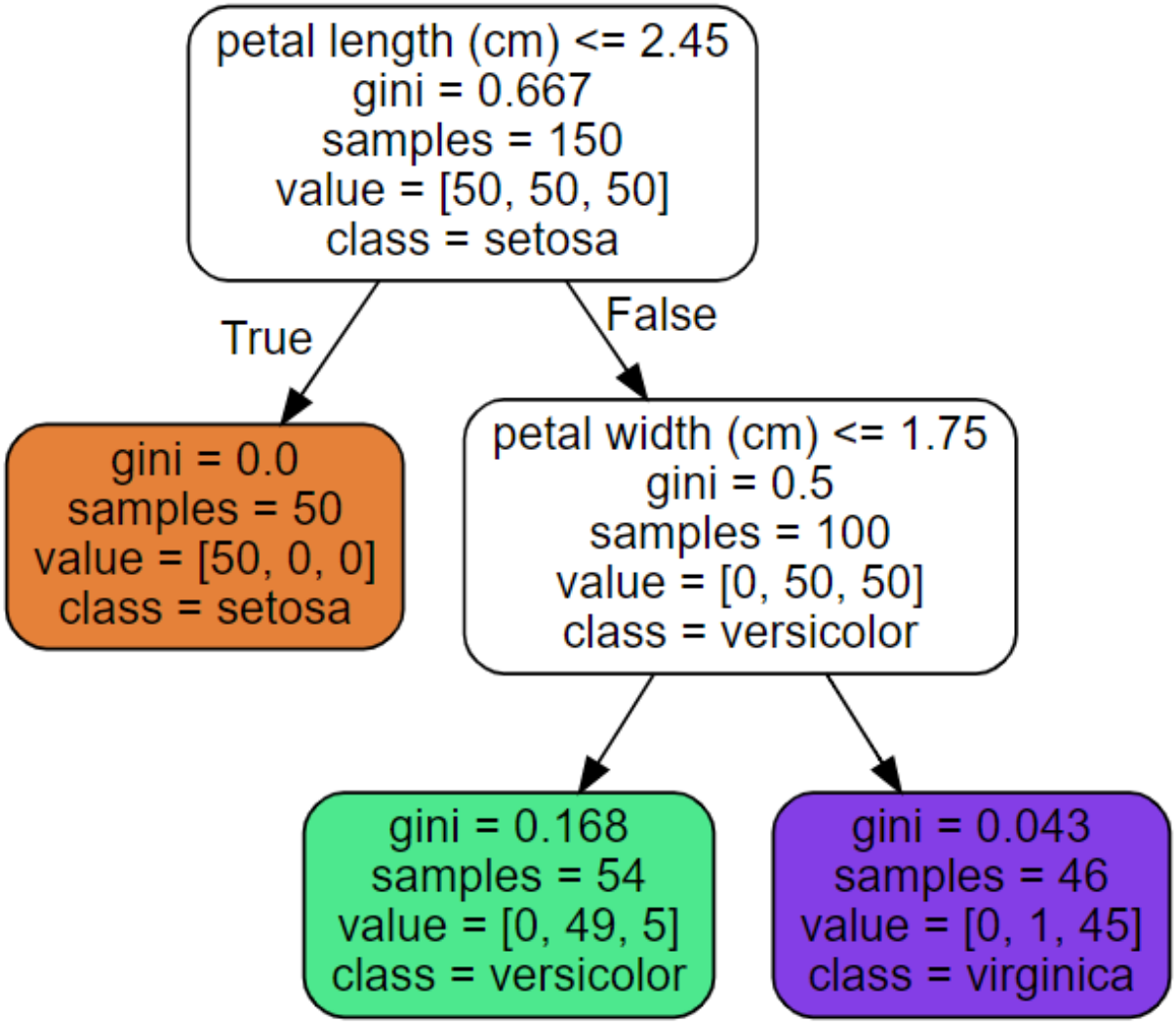
Como você pode ver, as Árvores de Decisão são bastante intuitivas e é fácil interpretar suas decisões. Esses modelos são geralmente chamados de *modelos de caixa branca*. Em contraste, como veremos, as Florestas Aleatórias ou as redes neurais são, geralmente, consideradas *modelos de caixa preta*. Elas fazem grandes previsões e você pode verificar facilmente os cálculos realizados para a obtenção delas; no entanto, é difícil explicar em termos simples por que essas previsões foram feitas. Por exemplo, se uma rede neural diz que uma pessoa específica aparece em uma imagem, é difícil saber o que realmente contribuiu para essa previsão: o modelo reconheceu os olhos dessa pessoa? Sua boca? Seu nariz? Os sapatos? Ou mesmo o sofá em que ela estava sentada? Por outro lado, as Árvores de Decisão fornecem regras simples de classificação que podem ser aplicadas mesmo manualmente se necessário (por exemplo, para a classificação de uma flor).

Algoritmo CART para treino de árvores

O Scikit-Learn utiliza o algoritmo CART, que produz somente *árvores binárias*: os nós sem folhas sempre têm dois filhos (ou seja, as perguntas somente terão respostas sim/não). Entretanto, outros algoritmos, como o ID3, podem produzir Árvores de Decisão com nós que têm mais de dois filhos.

- (há outros algoritmos para treinar árvores: ID3, C4.5 de Quinlan, etc.)
- Estes algoritmos baseiam-se em uma medida da "impureza" em cada nó da árvore (gini, entropia, etc.)

Recordando o exemplo



O atributo `samples` de um nó conta a quantidade de instâncias de treinamento a que se aplica. Por exemplo, 100 instâncias de treinamento têm um comprimento de pétala maior que 2,45cm (profundidade 1, direita) dentre as quais 54 têm uma largura de pétala menor que 1,75cm (profundidade 2, esquerda). O atributo `value` de um nó lhe diz a quantas instâncias de treinamento de cada classe este nó se aplica: por exemplo, o nó inferior direito se aplica a 0 Iris-Setosa, 1 Iris-Versicolor e 45 Iris-Virginica. Finalmente, o atributo `gini` de um nó mede sua *impureza*: um nó é "puro" (`gini=0`) se todas as instâncias de treinamento pertencem à mesma classe a qual se aplica. Por exemplo, uma vez que o nó esquerdo de profundidade 1 aplica-se apenas às instâncias de treinamento de Iris-Setosa, ele é puro e sua pontuação `gini` é 0. A Equação 6-1 mostra como o algoritmo de treinamento calcula a pontuação G_i do i-*ésimo* nó. Por exemplo, o nó esquerdo de profundidade 2 tem uma pontuação `gini` igual a $1 - (0/54)^2 - (49/54)^2 - (5/54)^2 \approx 0.168$. Outra *medida de impureza* será discutida brevemente.

Equação 6-1. Coeficiente de Gini

$$G_i = 1 - \sum_{k=1}^{n} p_{i,k}^{\ 2}$$

gini = 0.168
samples = 54
value = [0, 49, 5]
class = versicolor

```
occ = [0 49 5];
prob = occ / sum(occ);

gini = 1 - sum(prob.^2)
```

- $p_{i,k}$ é a média das instâncias da classe k entre as instâncias de treinamento no nó i.

Entropia

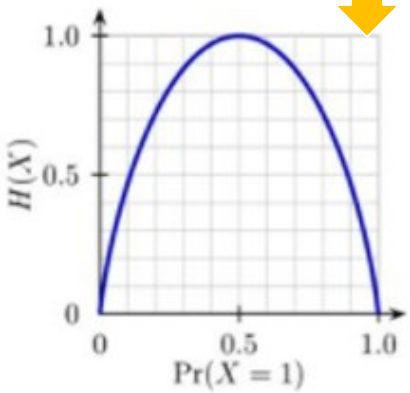
- Shannon entropy

$$H(p) = -\sum_{i=1}^{k} p(i) \log p(i),$$

- Binary entropy formula

$$H_b(p) = H(p, 1-p) = -p \log p - (1-p) \log(1-p)$$

Entropia $H_b(p)$ quando há duas classes apenas e log é na base 2

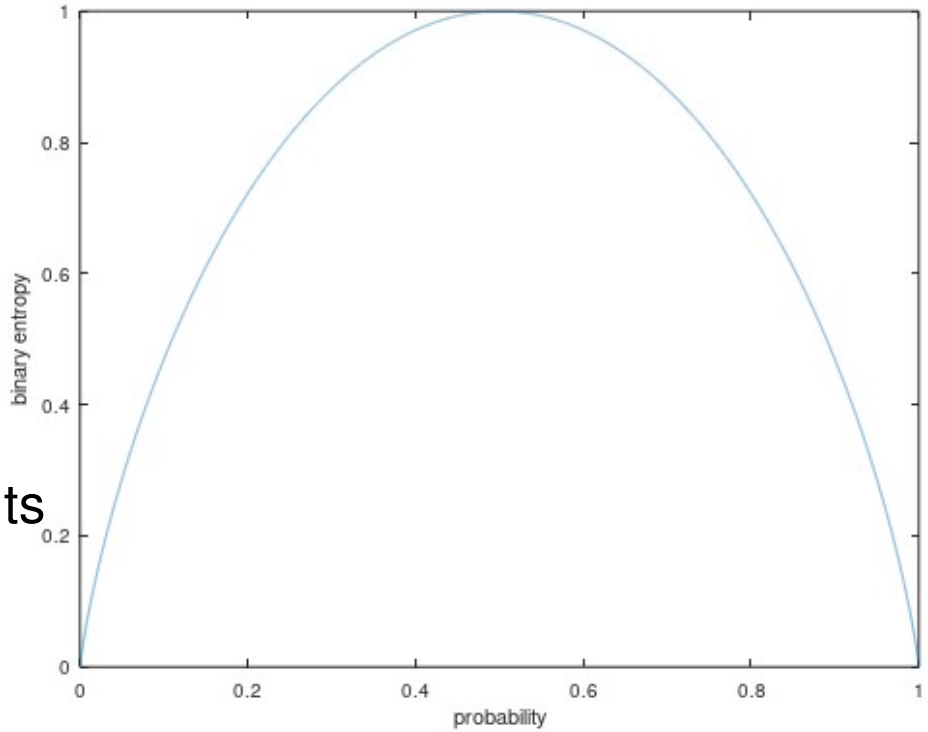


- Pode-se usar qualquer base: log (neperiano), log2 (base 2) ou log10
- Assume que há n=2 classes, cada uma com probabilidade P(x)=1/2, e base b=2. Daí entropia H(x)=1 bit

$$\begin{aligned} \mathrm{H}(X) &= -\sum_{i=1}^{n} \mathrm{P}(x_i) \log_b \mathrm{P}(x_i) \\ &= -\sum_{i=1}^{2} \frac{1}{2} \log_2 \frac{1}{2} \\ &= -\sum_{i=1}^{2} \frac{1}{2} \cdot (-1) = 1 \end{aligned}$$

Informação segundo Shannon

- Se algo tem probabilidade p, sua informação I é
  - $I = \log_2 (1 / p)$ bits
- Exemplo:
  - a) p = ¼ ➔
    - $I = \log_2(1 / (1/4)) = 2$ bits
  - b) p = 1/3 ➔
    - $I = \log_2(1 / (1/3)) = 1.585$ bits




```
%binary entropy
prob=linspace(0,1,10000);
information = log2(1 ./ prob); %information per class
entropy  = prob .* log2(1 ./ prob) + (1-prob) .* log2(1 ./ (1-prob));
entropy(isnan(entropy)) = 0; %Entropy
plot(prob, entropy)
xlabel('probability'), ylabel('binary entropy (bits)')
```

Coeficiente de Gini ou Entropia?

gini = 0.168
samples = 54
value = [0, 49, 5]
class = versicolor

A medida do *coeficiente* de Gini é aplicada por padrão, mas você pode selecionar a medida de impureza da *entropia* ao configurar o hiperparâmetro `criterion` para "`entropy`". O conceito de entropia originou-se na termodinâmica como uma medida da desordem molecular: a entropia aproxima-se de zero quando as moléculas ainda estão paradas e bem ordenadas. Mais tarde, se espalhou em uma ampla variedade de campos do conhecimento, incluindo a *Teoria da Informação* de Shannon, na qual mede o conteúdo médio de informação de uma mensagem:[4] a entropia é zero quando todas as mensagens são idênticas. No Aprendizado de Máquina, ela é frequentemente utilizada como uma medida do coeficiente: a entropia de um conjunto é zero quando contém instâncias de apenas uma classe. A Equação 6-3 mostra a definição da entropia do *i-ésimo* nó. Por exemplo, o nó esquerdo de profundidade 2 na Figura 6-1 possui uma entropia igual a

$$-\frac{49}{54}\log\left(\frac{49}{54}\right)-\frac{5}{54}\log\left(\frac{5}{54}\right)\approx 0.31$$

Equação 6-3. Entropia

$$H_i = -\sum_{\substack{k=1\\ p_{i,k}\neq 0}}^{n} p_{i,k}\log\left(p_{i,k}\right)$$

```
occ = [0 49 5]; #number of occurrences
prob = occ / sum(occ); %probability of each class
gini = 1 - sum(prob.^2) %gini
gini = 0.16804 %Gini index for this example
information = log(1 ./ prob) %information per class
weighted_information  = prob .* information
weighted_information(isnan(weighted_information)) = 0
entropy = sum(weighted_information) %Entropy
entropy = 0.30850 %Entropy for this example
```

Note que p log(1/p) é zero quando p=0

Entropia é usada em treino de árvores como parte do “ganho de informação”

- Entropia (livro do Géron usa log neperiano, mas adotamos abaixo o log na base 2)

$$H(Y) = -\sum_{y \in \mathcal{Y}} p(y) \log_2 p(y)$$

- Entropia condicional

$$H(Y|X_i) = -\sum_{x_i \in \mathcal{X}} p(x_i) \sum_{y \in \mathcal{Y}} p(y|x_i) \log_2 p(y|x_i)$$

- Ganho de informação (“information gain”): diferença entre entropias do nó pai e filhos

$$I(X_i; Y) = H(Y) - H(Y|X_i)$$

O Algoritmo de Treinamento CART

O Scikit-Learn utiliza o algoritmo da *Árvore de Classificação e Regressão* (CART, em inglês) para treinar árvores de decisão (também chamadas de árvores "*em crescimento*"). A ideia é realmente bastante simples: o algoritmo primeiro divide o conjunto de treinamento em dois subconjuntos utilizando uma única característica k e um limiar t_k (por exemplo, "comprimento da pétala ≤ 2,45cm"). Como ele escolhe k e t_k? Ele busca pelo par (k, t_k) que produz os subconjuntos mais puros (ponderados pelo tamanho). A função de custo que o algoritmo tenta minimizar é dada pela Equação 6-2.

Equação 6-2. Função de custo CART para classificação

$$J(k,t_k) = \frac{m_{\text{esquerda}}}{m} G_{\text{esquerda}} + \frac{m_{\text{direita}}}{m} G_{\text{direita}}$$

sendo que $\begin{cases} G_{\text{esquerda/direita}} & \text{mede a impureza do subconjunto esquerdo/direito,} \\ m_{\text{esquerda/direita}} & \text{é o número de instâncias no subconjunto esquerdo/direito.} \end{cases}$

Depois de dividir com sucesso o conjunto de treinamento em dois, ele divide os subconjuntos utilizando a mesma lógica, depois os subsubconjuntos e assim por diante, recursivamente. Ele para de recarregar uma vez que atinge a profundidade máxima

Custo para treino e teste

Complexidade Computacional

Fazer previsões requer percorrer a Árvore de Decisão da raiz até uma folha. Em geral, as árvores de decisão são equilibradas, de modo que percorrê-la requer passar por aproximadamente $O(log_2(m))$ nós.[3] Uma vez que cada nó exige apenas a verificação do valor de uma característica, a complexidade geral da previsão será apenas $O(log_2(m))$, independentemente do número de características. Então, as previsões são muito rápidas mesmo se tratando de grandes conjuntos de treinamento.

No entanto, o algoritmo de treinamento compara todas as características (ou menos, se `max_features` estiver definido) em todas as amostras a cada nó. Isto resulta em uma complexidade de treinamento $O(n \times m\ log(m))$. Ao pré-selecionar os dados para pequenos conjuntos (menos de algumas milhares de instâncias), o Scikit-Learn pode acelerar o treinamento (defina `presort = True`), mas isso diminui significativamente o treinamento para conjuntos maiores.

(definida pelo hiperparâmetro `max_depth`) ou se não consegue encontrar uma divisão que reduza a impureza. Alguns outros hiperparâmetros (que já serão descritos) controlam as condições adicionais de parada (`min_samples_split`, `min_samples_leaf`, `min_weight_fraction_leaf` e `max_leaf_nodes`).

Como você pode ver, o algoritmo CART é um *algoritmo ganancioso*: ele busca gananciosamente uma divisão otimizada no nível superior e, em seguida, repete o processo em cada nível. Ele não verifica se a divisão irá ou não resultar na menor impureza possível nos vários níveis abaixo. Um algoritmo ganancioso geralmente produz uma solução razoavelmente boa, mas não é garantia de uma solução ideal.

Infelizmente, encontrar a árvore ideal é conhecido por ser um problema *NP-Completo*:[2] ele requer tempo $O(\exp(m))$, tornando o problema intratável mesmo para conjuntos de treinamento muito pequenos. É por isso que devemos nos contentar com uma solução "razoavelmente boa".

Hiperparâmetros de Regularização

As Árvores de Decisão fazem poucos pressupostos sobre os dados de treinamento (ao contrário dos modelos lineares, que obviamente assumem que os dados são lineares, por exemplo). Se for deixada sem restrições, a estrutura da árvore se adaptará aos dados de treinamento muito de perto, provavelmente se sobreajustando. Tal modelo geralmente é chamado de *modelo não paramétrico*, não porque ele não tenha quaisquer parâmetros (geralmente ele têm bastante), mas porque o número de parâmetros não é determinado antes do treinamento, então a estrutura do modelo é livre para ficar próxima aos dados.

Em contraste, um *modelo paramétrico,* como um modelo linear, tem um número predeterminado de parâmetros, então seu grau de liberdade é limitado, reduzindo o risco de sobreajuste (mas aumentando o risco de subajuste).

Para evitar o sobreajuste nos dados de treinamento, você precisa restringir a liberdade da Árvore de Decisão durante este treinamento. Como você sabe, isso é chamado de regularização. Os hiperparâmetros de regularização dependem do algoritmo utilizado, mas geralmente você pode, pelo menos, restringir a profundidade máxima da Árvore de Decisão. No Scikit-Learn, isto é controlado pelo hiperparâmetro `max_depth` (o valor padrão é `None`, que significa ilimitado). Reduzir `max_depth` regularizará o modelo e reduzirá, assim, o risco de sobreajuste.

A classe `DecisionTreeClassifier` tem alguns outros parâmetros que restringem de forma semelhante a forma da Árvore de Decisão: `min_samples_split` (o número mínimo de amostras que um nó deve ter antes que possa ser dividido), `min_samples_leaf` (o número mínimo de amostras que um nó da folha deve ter), `min_weight_fraction_leaf` (o mesmo de `min_samples_leaf`, mas expressa como uma fração do número total de instâncias ponderadas), `max_leaf_nodes` (número máximo de nós da folha) e `max_features` (número máximo de características que são avaliadas para divisão em cada nó). Aumentar os hiperparâmetros `min_*` ou reduzir os hiperparâmetros `max_*` regularizará o modelo.

Crescer árvore e depois podar

Outros algoritmos funcionam primeiro treinando a Árvore de Decisão sem restrições, depois *podando* (eliminando) nós desnecessários. Um nó cujos filhos são todos nós da folha é considerado desnecessário caso a melhoria da pureza que ele fornece não seja *estatisticamente significante*. Os testes estatísticos padrão, como o *teste* χ^2, são utilizados para estimar a probabilidade de que a melhora seja puramente o resultado do acaso (que é chamada de *hipótese nula*). Se essa probabilidade, denominada *p-value*, for superior a um determinado limiar (geralmente 5%, controlado por um hiperparâmetro), então o nó é considerado desnecessário e seus filhos são excluídos. A poda continua até que todos os nós desnecessários tenham sido eliminados.

Efeito de regularização em classificação

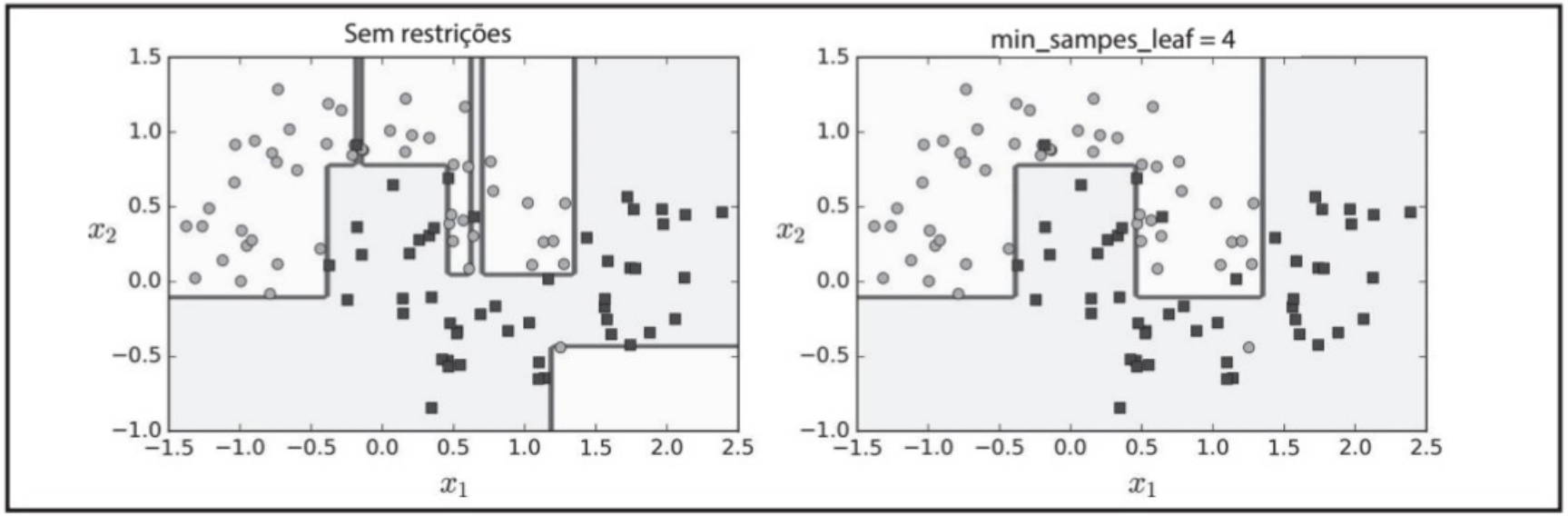


Figura 6-3. Regularização utilizando min_samples_leaf

Opções

- Classificação
- Estimativa de probabilidade de cada classe
- Regressão

Mais do que decider a classe: estimar a probabilidade de cada classe

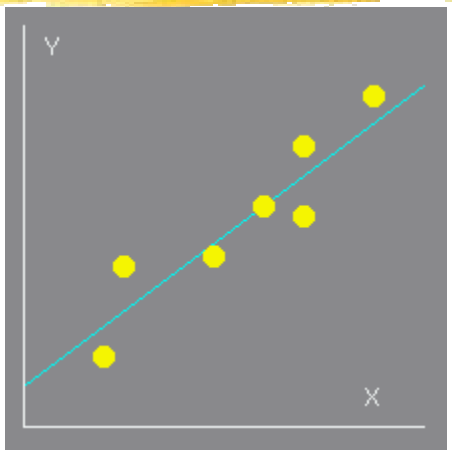
Estimando as Probabilidades de Classes

Uma Árvore de Decisão também pode estimar a probabilidade de uma instância pertencer a uma classe específica *k*: primeiro ela atravessa a árvore para encontrar o nó da folha para esta instância e, em seguida, retorna a taxa das instâncias de treinamento da classe *k* neste nó. Por exemplo, suponha que ela tenha encontrado uma flor cujas pétalas têm 5cm de comprimento e 1,5cm de largura. O nó da folha correspondente é o nó esquerdo de profundidade 2, portanto a Árvore de Decisão deve exibir as seguintes probabilidades: 0% para Iris-Setosa (0/54), 90,7% para Iris-Versicolor (49/54) e 9,3% para Iris-Virginica (5/54). E, claro, se você pedir que ela preveja a classe, ela deve exibir Iris-Versicolor (classe 1), pois é a que tem a maior probabilidade. Verificaremos isso:

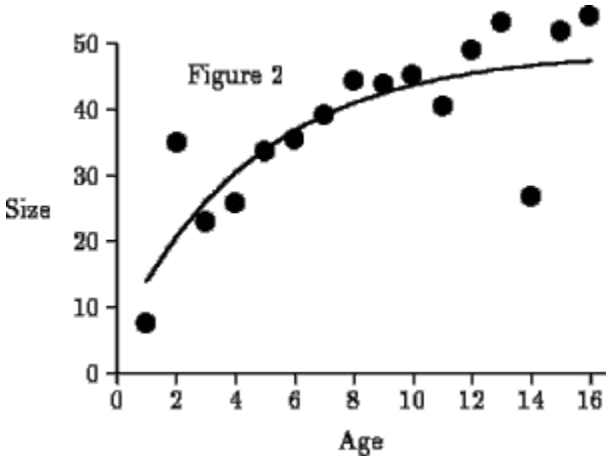
```
>>> tree_clf.predict_proba([[5, 1.5]])
array([[ 0. ,  0.90740741,  0.09259259]])
>>> tree_clf.predict([[5, 1.5]])
array([1])
```

Predição (ou regressão)

- Regressão linear



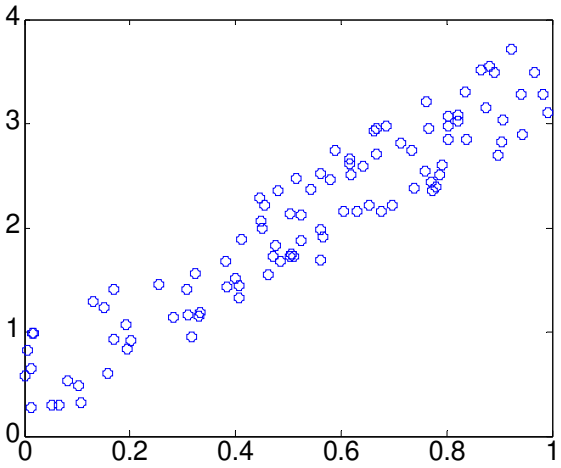
- Regressão não-linear



Regressão linear

- Código Matlab

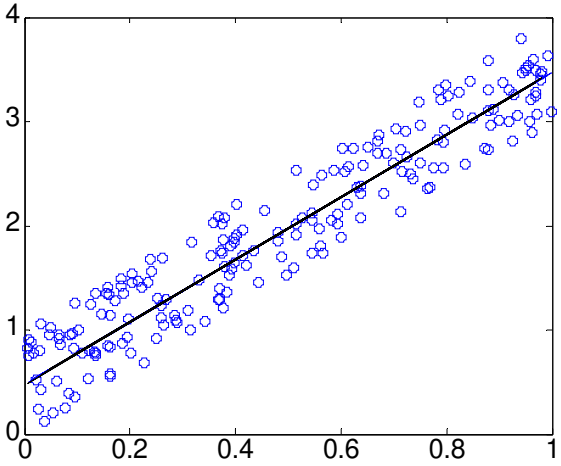
```
N=100; a=3;
t=rand(1,N);
x=a*t+rand(1,N);
plot(t,x,'o');
```



- Linear Regression Model (Weka)

Y = 2.9974 * X + 0.4769

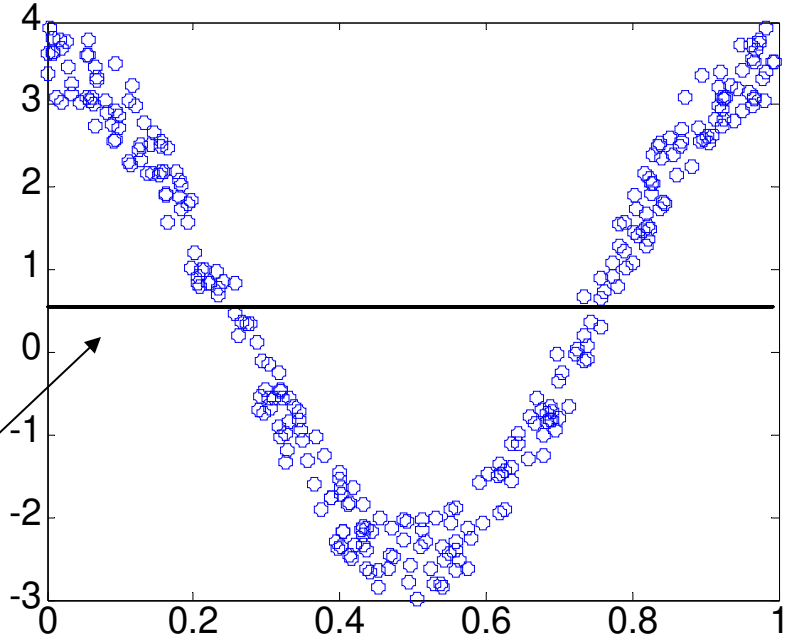
Correlation coefficient	0.9358
Mean absolute error	0.2744



Regressão não-linear

- Código Matlab

```
N=100;
a=3;
x=rand(1,N);
y=a*cos(2*pi*x)+rand(1,N);
plot(x,y,'o');
```



- Problema:
  - Linear não resolve
- Solução:
  - Redes neurais
  - SVM

Regressão

As Árvores de Decisão também desempenham tarefas de regressão. Construiremos uma árvore de regressão utilizando a classe `DecisionTreeRegressor` do Scikit-Learn e treiná-la em um conjunto de dados quadrático confuso com `max_depth=2`:

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = DecisionTreeRegressor(max_depth=2)
tree_reg.fit(X, y)
```

A árvore resultante está representada na Figura 6-4.

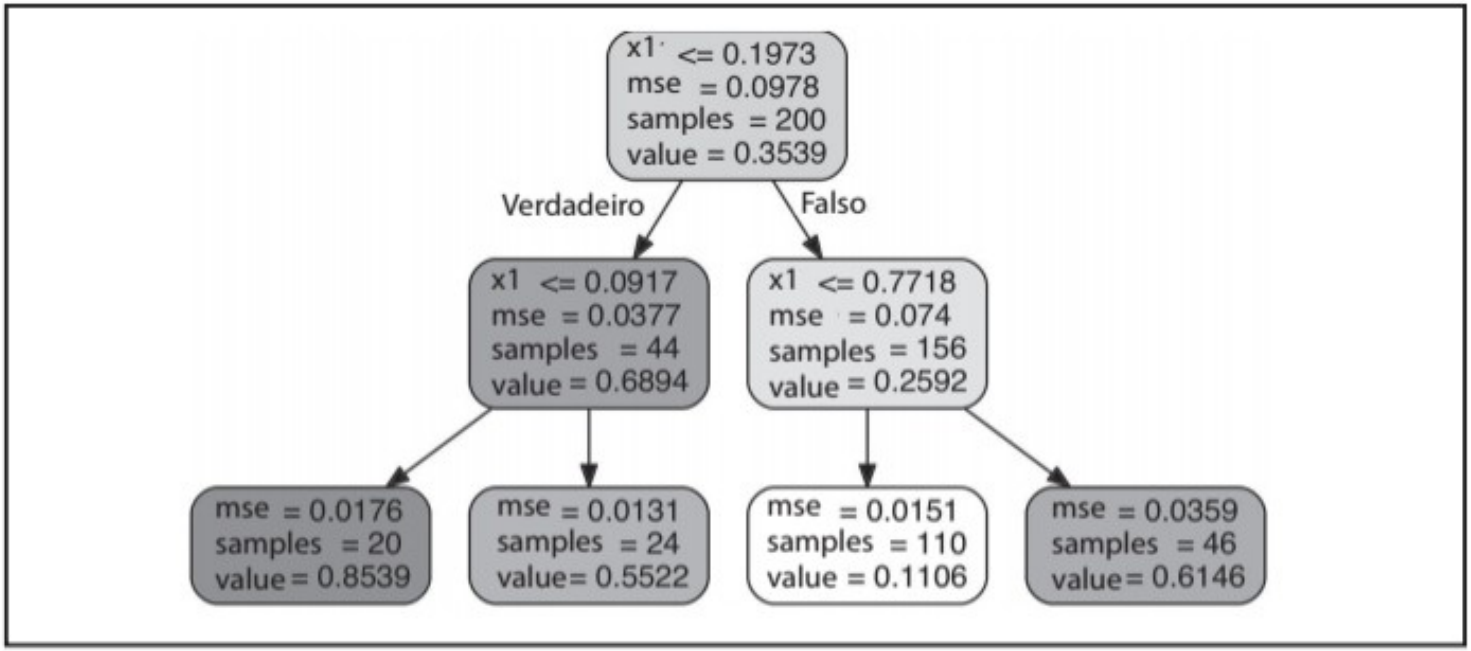


Figura 6-4. Uma Árvore de Decisão para regressão

CART para regressão

O algoritmo CART funciona mais ou menos da mesma forma, exceto que, em vez de tentar dividir o conjunto de treinamento de forma a minimizar a impureza, ele agora tenta dividi-lo de forma a minimizar o MSE. A Equação 6-4 mostra a função de custo que o algoritmo tenta minimizar.

Equação 6-4. A função CART de custo para regressão

$$J(k, t_k) = \frac{m_{\text{esquerda}}}{m}\text{MSE}_{\text{esquerda}} + \frac{m_{\text{direita}}}{m}\text{MSE}_{\text{direita}} \text{ sendo que } \begin{cases} \text{MSE}_{\text{nó}} = \sum_{i \in \text{nó}} \left(\hat{y}_{\text{nó}} - y^{(i)}\right)^2 \\ \hat{y}_{\text{nó}} = \frac{1}{m_{\text{nó}}} \sum_{i \in \text{nó}} y^{(i)} \end{cases}$$

Regularização para regressão

As previsões deste modelo estão representadas à esquerda da Figura 6-5. Se você definir `max_depth=3`, obterá as previsões representadas à direita. Observe como o valor previsto para cada região é sempre o valor-alvo médio das instâncias presentes nela. O algoritmo divide cada região de uma forma, o que faz com que a maior parte das instâncias de treinamento se aproxime o máximo possível desse valor previsto.

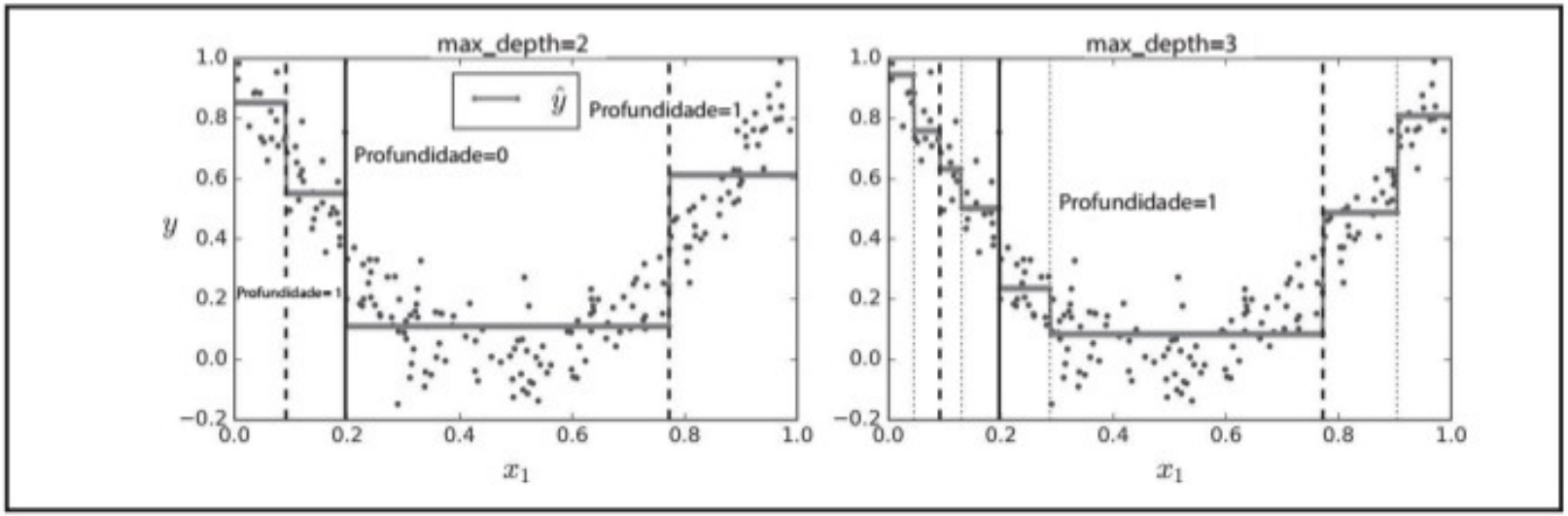


Figura 6-5. Previsões de dois modelos de regressão da Árvore de Decisão

+ regularização para regressão

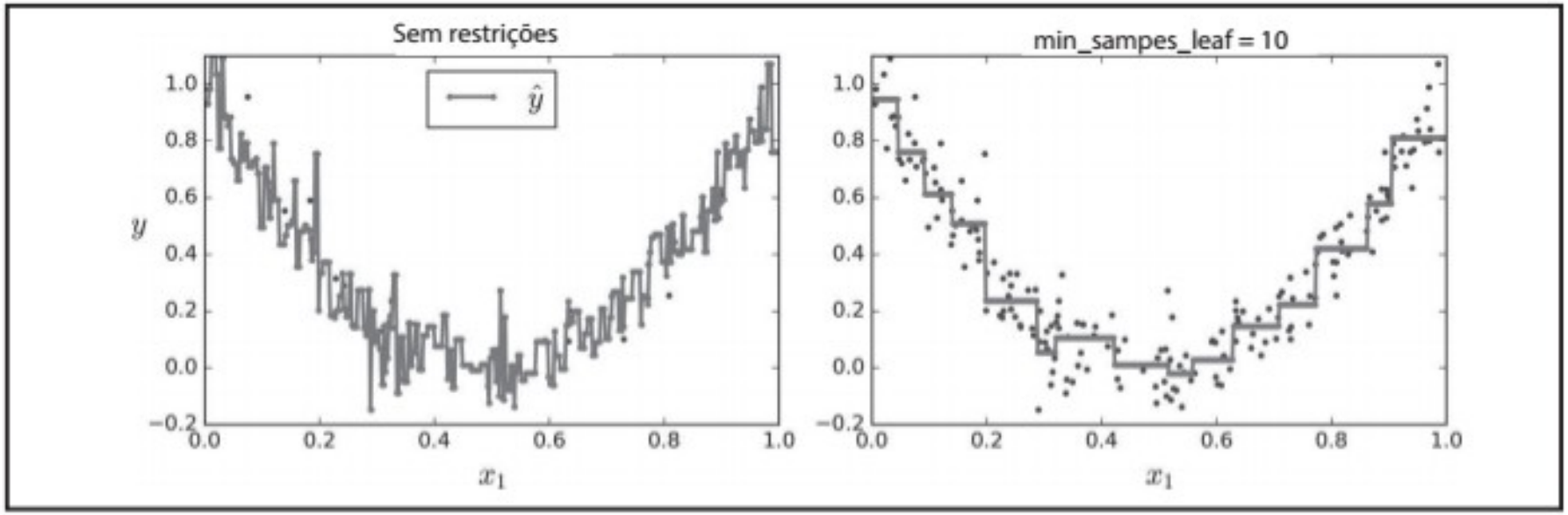
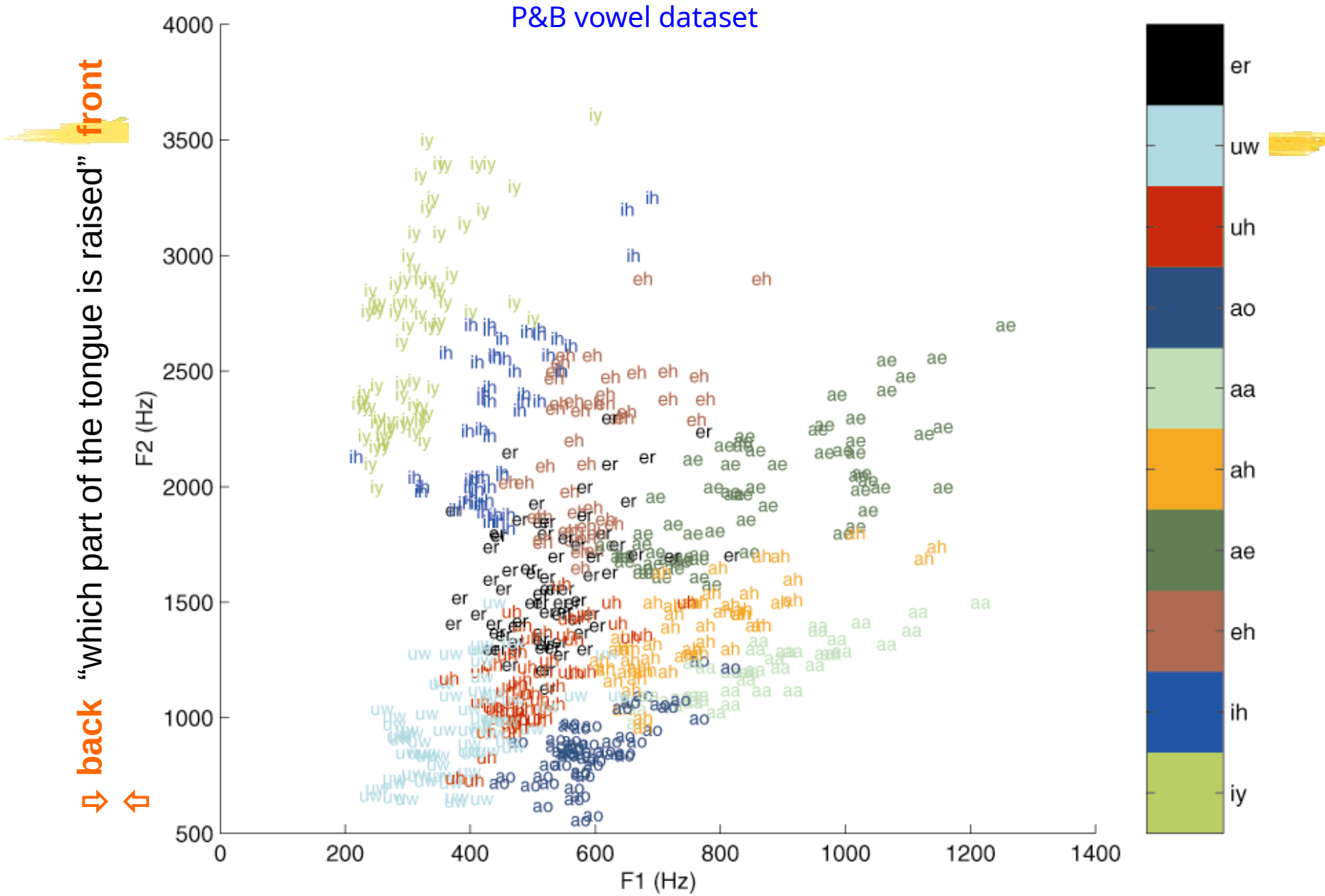


Figura 6-6. Regularizando um regressor da Árvore de Decisão

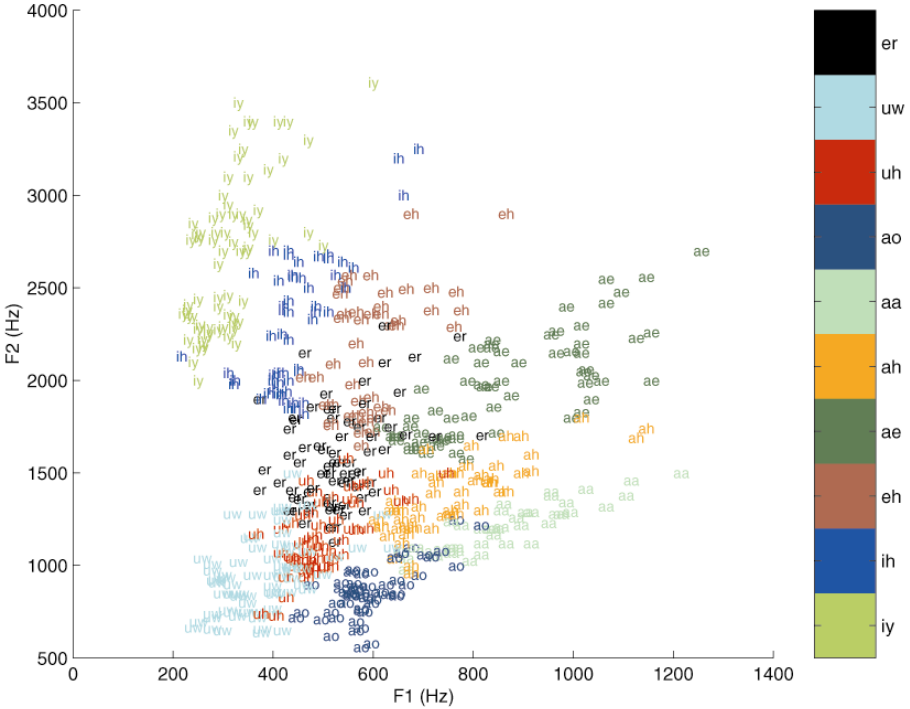
P&B vowel dataset



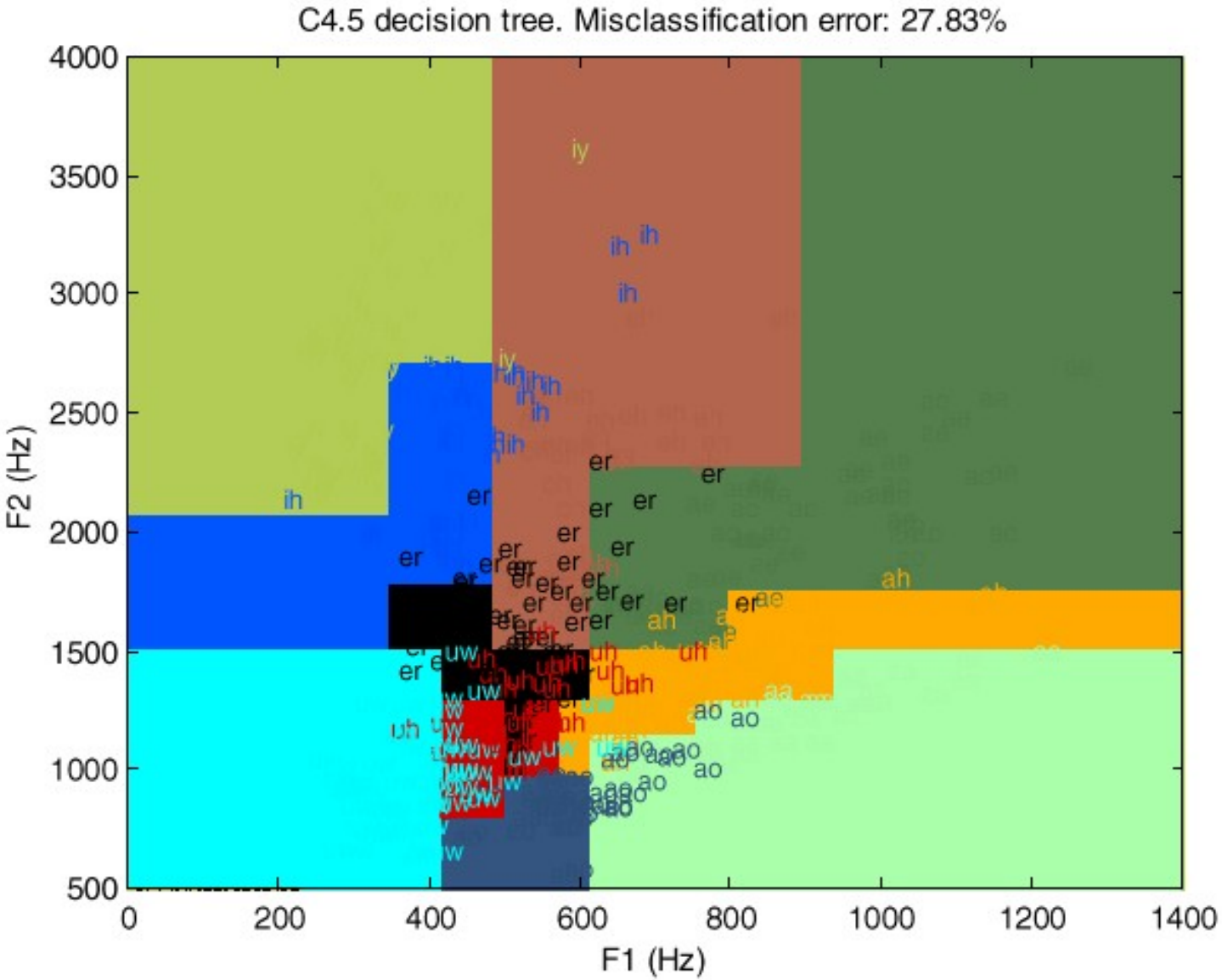
Árvore de decisão C4.5 (J4.8)

```
F2 <= 1600
|   F1 <= 586
|   |   F2 <= 990: UW (92.0/47.0)
|   |   F2 > 990
|   |   |   F2 <= 1200: UH (50.0/17.0)
|   |   |   F2 > 1200: ER (63.0/23.0)
|   F1 > 586
|   |   F2 <= 1250: AA (59.0/33.0)
|   |   F2 > 1250: AH (56.0/24.0)
F2 > 1600
|   F1 <= 490
|   |   F1 <= 350: IY (68.0/5.0)
|   |   F1 > 350: IH (61.0/21.0)
|   F1 > 490
|   |   F1 <= 652: EH (71.0/34.0)
|   |   F1 > 652: AE (79.0/19.0)
```

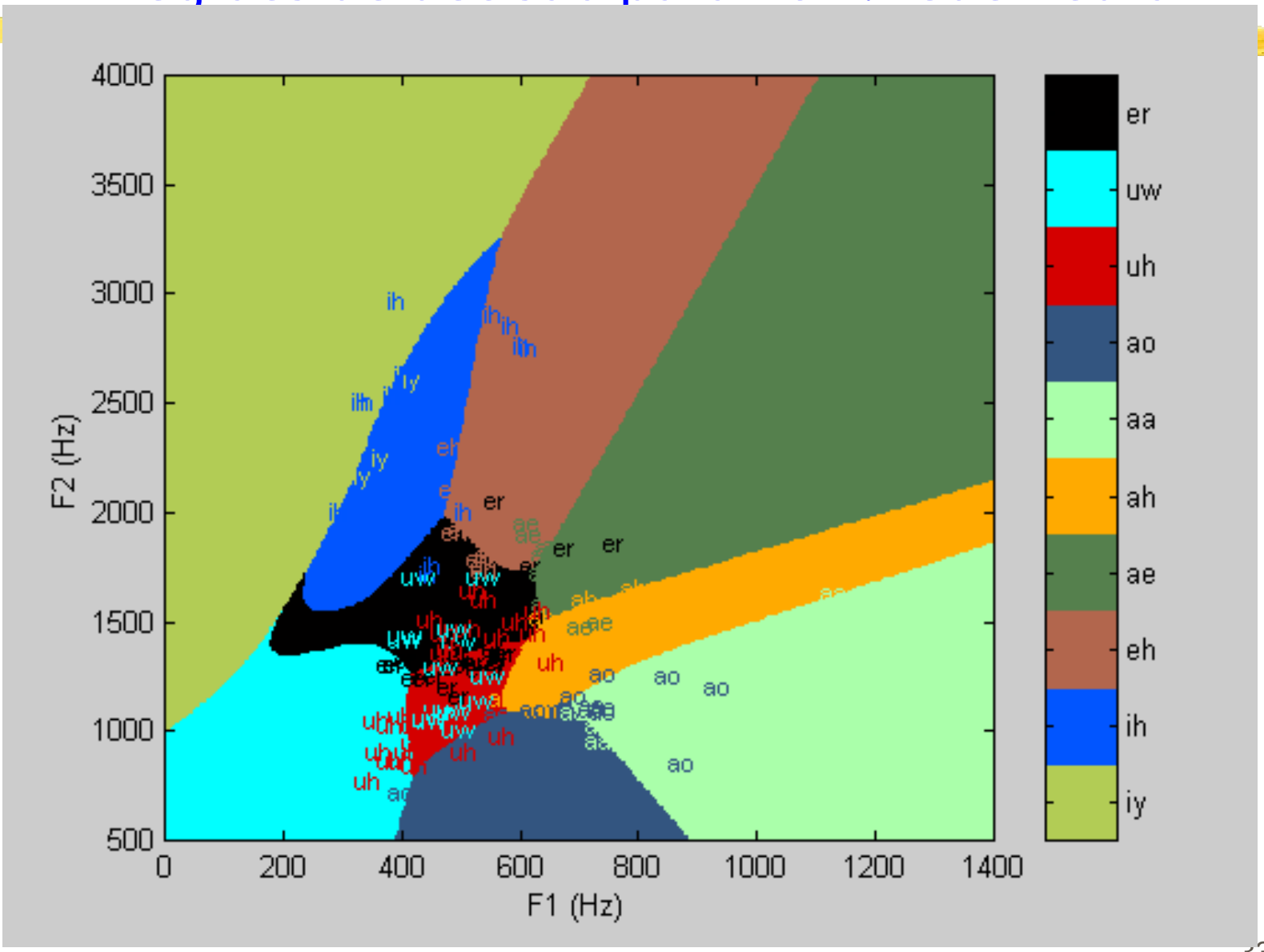
Tamanho: 17 nós e 9 folhas



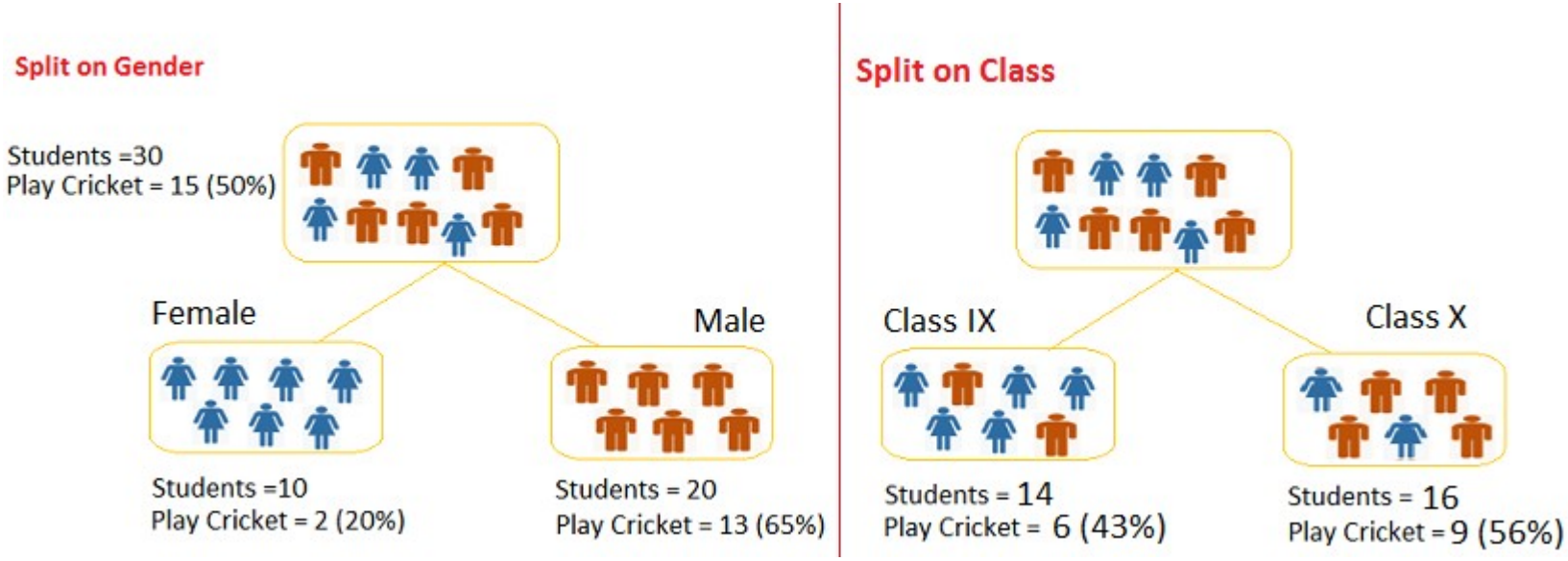
Exemplo de regiões de decisão



Regiões de decisão para P&B: rede neural



Como literatura discute árvores de decisão

Algo / Split Criterion	Description	Tree Type
Gini Split / Gini Index	Favors larger partitions. Very simple to implement.	CART
Information Gain / Entropy	Favors partitions that have small counts but many distinct values.	ID3 / C4.5

- Sites:
- https://medium.com/@rishabhjain_22692/decision-trees-it-begins-here-93ff54ef134
- http://www.learnbymarketing.com/481/decision-tree-flavors-gini-info-gain/